Mark	/ 30

Team name	**B14 – Altium League**
Project Title	GEOGUESSER probe as mouse pointer development and integration

1) **Each team member describes their individual contribution** (maximum 100 words per member).

- **Fabio Cambedda**: I mainly contributed to the development of the probe hardware using Altium Designer. My work included searching for and importing IC, resistor and capacitor models, verifying and correcting footprint layers and designator assignments, linking schematic symbols to PCB footprints and drawing the circuit schematics. I was also deeply involved in the PCB design process, optimizing component placement and organization, routing tracks and vias, and managing the ground pour to ensure an efficient layout. Finally, I contributed to the review and revision of the project report, helping to improve its technical accuracy and overall quality.

- **Sofia Capozzi del Pozo**: My primary contribution focused on developing the hardware probe using Altium Designer. I managed the full hardware lifecycle, from the schematic to the PCB layout. After analyzing the IMU and magnetometer datasheets, I selected the appropriate components and verified their footprints. During the PCB design phase, I optimized the component placement to fit the required form factor. Finally, I handled the routing process, strategically placing tracks and vias to ensure an efficient layout, focusing on proper power distribution and signal integrity.

- **Pasquale Ludovico Ignarra**: My role was to lead and coordinate the team, assigning tasks and reviewing the results. For the PCB design, I analyzed the possible sensors and selected the IMU and magnetometer. I also checked the schematic connections and component placement in Altium, and proposed solid ground pours and via stitching to reduce noise. For the software part, I identified the sensor addresses required for MCU communication and defined the overall program functionality. Finally, I coordinated the report writing by collecting, organizing, and integrating the contributions of all team members.

- **Alessandro Lima**: My main contribution focused on firmware and software development for the smart wearable system. I developed the STM32 firmware for sensor configuration, real-time data acquisition, and BLE communication between the probe and the smartphone application. I implemented the IMU and magnetometer drivers, the data packet structure, and the acquisition logic based on interrupts and I2C communication. In addition, I developed a Flutter application that is used to visualize sensor data, display real-time graphs, and implement a head-controlled cursor system with gesture-based navigation between pages. I also contributed to system integration, debugging, and optimization of communication between hardware and software components.

- **Andrea Toselli**: After the initialization of magnetometer development, my main contribution was focused on firmware and software development for the smart wearable system. I supported the development of the STM32 firmware for sensor configuration, real-time data acquisition, and BLE communication between the probe and the smartphone application. I implemented IMU drivers, the data packet structure, and acquisition logic based on interrupts and I2C. In addition, I supported the implementation of a Flutter application to visualize sensor data, display real-time graphs, and implement a head-controlled cursor system with gesture-based navigation. Finally, I provided secondary support for system integration, debugging, and hardware-software optimization.

2) **Description of the project aims** (maximum 200 words).

Pasquale Ludovico Ignarra (30%), Sofia Capozzi (20%), Fabio Cambedda (15%), Andrea Toselli (15%), Alessandro Gabriela Lima (15%)

Project objectives

The aim of this project is to design and develop the PCB of a GEOGUESSER probe to be integrated into smart glasses, together with the firmware required to use the device as a motion-based mouse pointer.

The idea originates from a preliminary literature review, which showed that inertial sensors embedded in smart glasses are mainly used in medical and healthcare applications, for example for head posture assessment and neck-disorder monitoring. Starting from this observation, the project explores a different use case: exploiting head-motion information acquired by the probe to control a pointer interface.

The proposed system can also be interpreted as a lightweight alternative for eye-tracking-related applications. Conventional eye-tracking solutions are usually based on infrared cameras and computer vision algorithms, which provide high accuracy but often require significant computational resources and power consumption. In contrast, this project investigates a simpler and less power-demanding solution based on inertial sensing, aiming to provide an efficient human–machine interaction method for wearable devices. At the current stage, the system has been validated through a mobile application, where the probe controls a pointer-like interface.

3) **Description of the hardware designed for the project**, including a block diagram, schematics, layouts and cost for production (maximum 500 words + figures).

Pasquale Ludovico Ignarra (70%), Sofia Capozzi (90%), Fabio Cambedda (90%), Andrea Toselli (10%), Alessandro Gabriela Lima (10%)

Hardware description

The probe integrates two sensing devices: the LSM6DSV16BX IMU and the IIS2MDC magnetometer. The barometer was discarded since its measurements were not relevant to the application, reducing both power consumption and board complexity. The probe is connected to the main board through a flex-bridge interface. To ensure compatibility with the BM29B-24DP/2-0.35V(51) connector mounted on the flex board, the corresponding BM29B-24DS/2-0.35V(51) receptacle was selected.

Figure 1: IMU, magnetometer and connector selected

Decoupling capacitors were placed close to the sensor supply pins to improve power-supply stability and reduce high-frequency noise. The capacitor network was selected according to the datasheet recommendations, while prioritizing compact package sizes to minimize PCB area occupation. Although I^2C pull-up resistors are already available on the main board, they were also included on the probe to increase the portability and reusability of the design.

The overall flow of information is represented by the following Block Scheme:

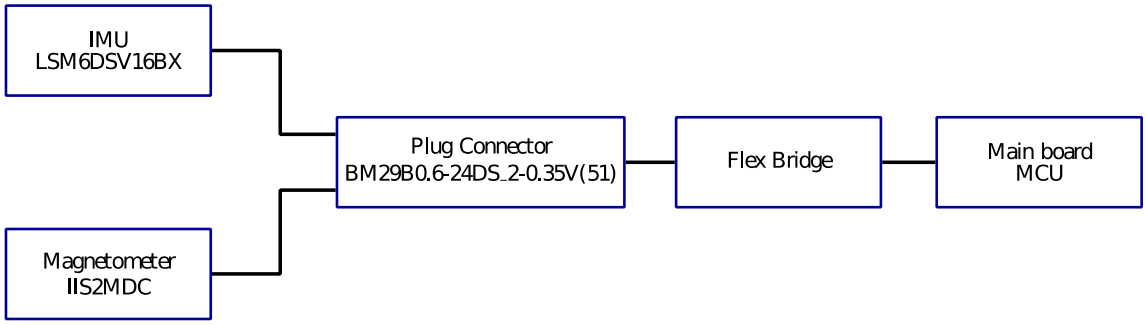


Figure 2: Block scheme

The complete electrical connections are reported in the schematic sheet

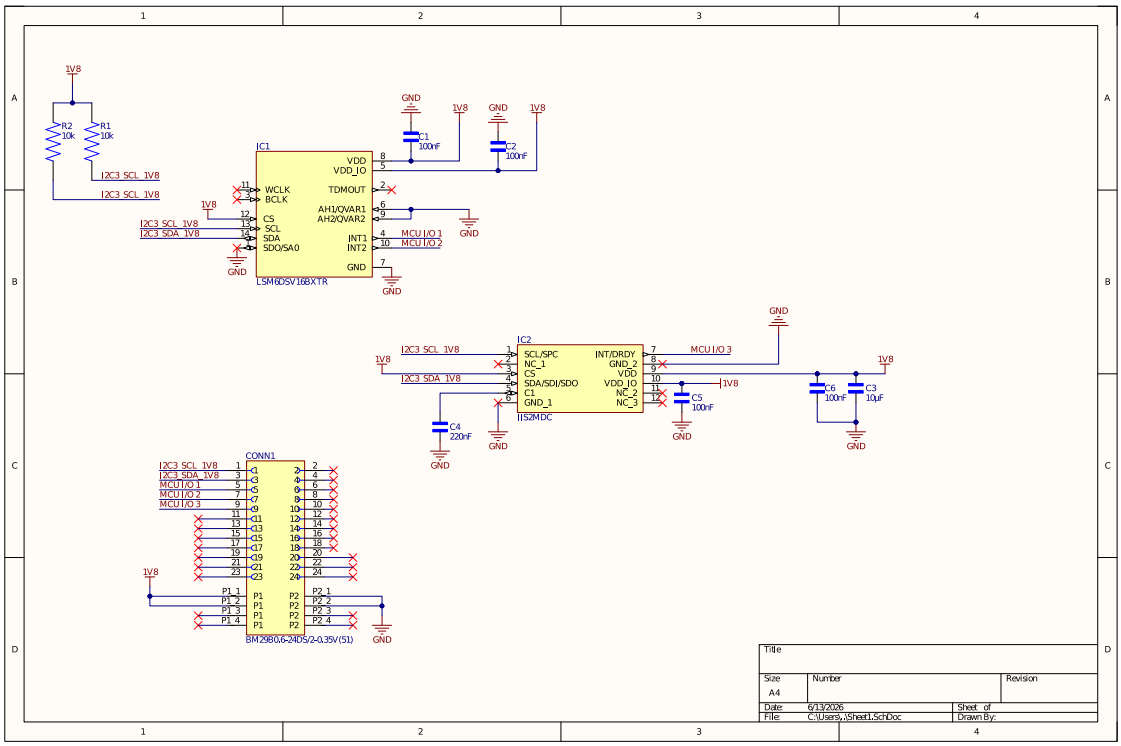


Figure 3: Probe schematic sheet

Remarks

Both sensors support supply voltages from 1.71 V to 3.6 V. Therefore, a 1.8 V supply was selected to reduce power consumption. Thanks to the level shifters available on the main board, the I²C logic levels are also set to 1.8 V. Although not all component names are reported in the schematic, the Bill of Materials (BOM) contains the complete list of selected parts, allowing the probe to be directly reproduced through PCB manufacturing services such as JLCPCB or PCBWay. No ERC directives were added only to suppress warnings related to specific ground pins and to the unused pins on the right side of the connector during project validation.

PCB layout

A two-layer PCB was selected because the design requires only a limited number of signals. Both copper layers were used as signal layers for routing purposes. FR-4 was chosen as dielectric material because it is the standard solution for rigid PCBs, while the thickness was set to 0.8 mm to satisfy manufacturing constraints and reduce production costs.

#	Name	Material	Type	Weight	Thickness	Dk
	Top Overlay		Overlay			
	Top Solder	Solder Resist	Solder Mask		10.16µm	3.5
1	Top Layer		Signal	1oz	35.56µm	
	Dielectric 1	FR-4	Dielectric		708.56µm	4.8
2	Bottom Layer		Signal	1oz	35.56µm	
	Bottom Solder	Solder Resist	Solder Mask		10.16µm	3.5
	Bottom Overlay		Overlay			

Figure 4: Probe stackup

The design rules were set according to the manufacuring and assembly capabilities of JLCPCB. Here the main parameters are listed:

Minimum via – track clearence: **0.102 mm**
Minimun track width: **0.152 mm**
Minimum via hole size: **0.711 mm**
Minimum via diameter: **1.27 mm**
Via Hole to Hole clearence: **0.254 mm**

The probe was designed with a compact rectangular shape suitable for integration into the temple area of smart glasses. The final dimensions are **11.303 mm × 12.319 mm**.
Sensors, passive components, and pull-up resistors are placed on the top layer, while the connector is mounted on the bottom layer.

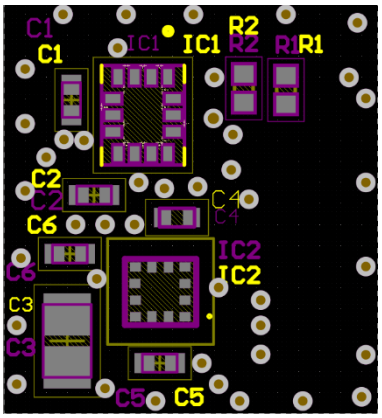


Figure 5a: Probe top layer

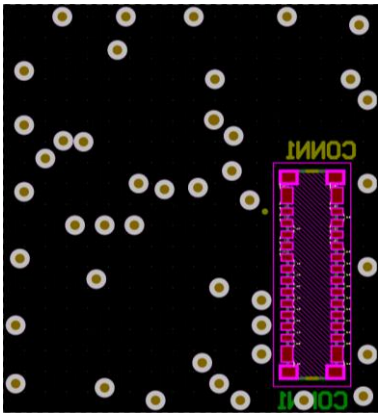


Figure 5b: Probe bottom layer

To compensate for the absence of a dedicated ground plane, solid copper pours were added on both layers and connected through via stitching. This solution improves return-current paths, signal integrity, and thermal dissipation.

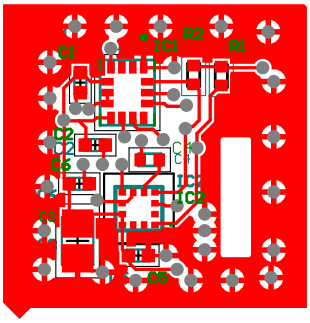


Figure 6a: Top layer copper pour

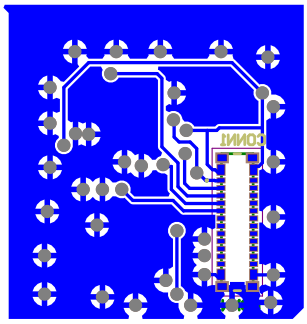


Figure 6b: Bottom layer copper pour

A final remark concerns the connector orientation. Due to the mirrored placement of the receptacle connector with respect to the plug connector on the flex bridge, the probe must be rotated by 180° to ensure proper matching and electrical connection.

PCB cost

Finally, Gerber, BOM, and Pick-and-Place files were generated and uploaded to JLCPCB for manufacturing estimation. The overall production cost is approximately **160 €**, excluding shipping costs.

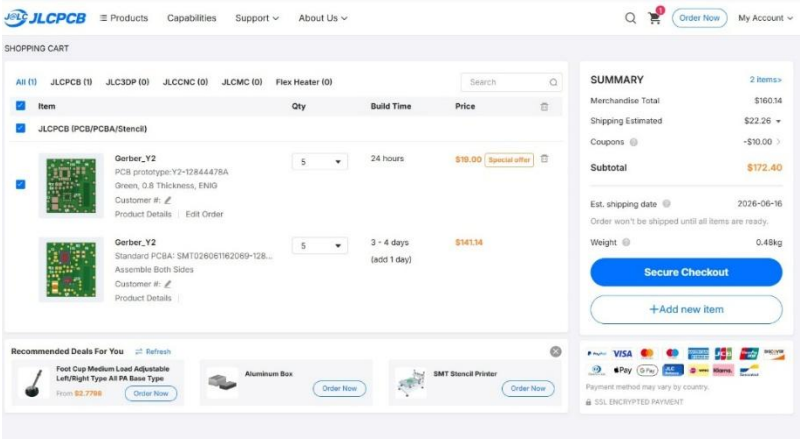


Figure 7: JLCPCB cart for PCB fabrication

4) **Description of the firmware developed** for sensor configuration, data acquisition, and data processing. This section should also include the code of the main functions developed by the team. (maximum 500 words + code).

Pasquale Ludovico Ignarra (40%), Sofia Capozzi (5%), Fabio Cambedda (5%), Andrea Toselli (50%), Alessandro Gabriela Lima (60%)

Firmware description

The firmware was developed for the STM32U575 microcontroller and was designed to manage sensor configuration, real-time data acquisition, BLE communication, and interaction with the smartphone application. Communication with the sensors is performed through the I²C protocol, while BLE communication is handled through an RN4871 module connected via UART. All main peripherals and MCU pins were configured on STM32CubeMX software, as reported in the images below.

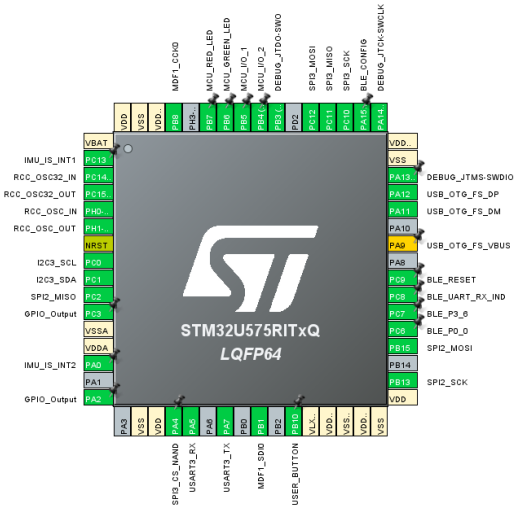


Figure 8: MCU pin assignment

I²C3 was configured in standard mode at 100 kHz.

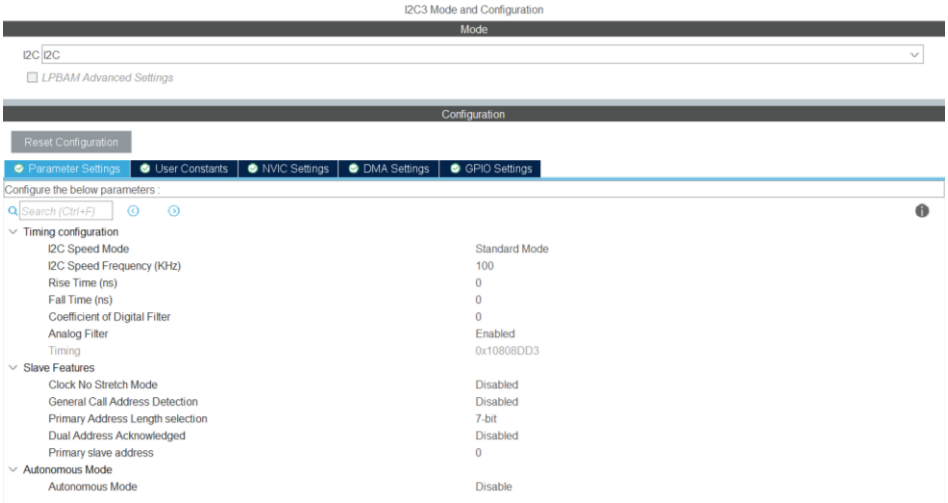
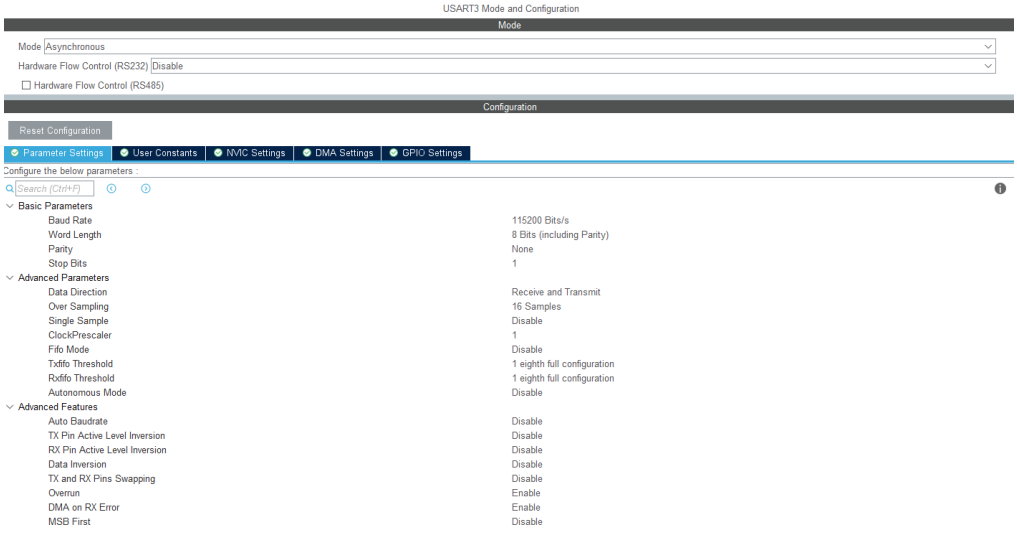


Figure 9: I2C configuration

USART3 was configured at 115200 bit/s with interrupt-based reception of smartphone commands.



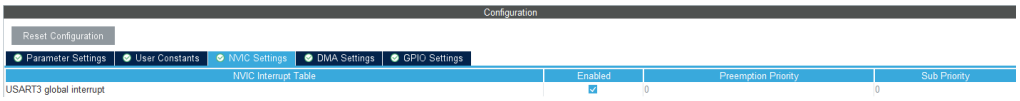


Figure 10: USART configuration

TIM2 generates periodic interrupts every 10 ms, providing a 100 Hz acquisition rate.

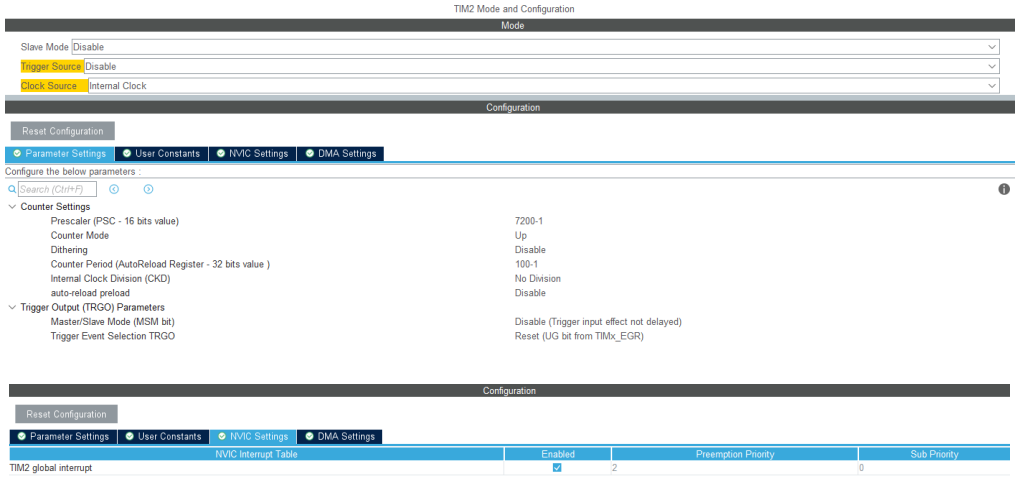


Figure 11: Timer configuration

Dedicated drivers were developed for the LSM6DSV16BX IMU and the IIS2MDC magnetometer.

ext_imu_driver.h

```
#ifndef __EXT_IMU_DRIVER_H__
#define __EXT_IMU_DRIVER_H__

#include <stdint.h>
#include "main.h"
#define EXT_IMU_I2C_ADDR         (0x6A << 1)
#define EXT_IMU_WHO_AM_I_REG     0x0F
#define EXT_IMU_WHO_AM_I_VAL     0x71
#define EXT_IMU_CTRL1_XL         0x10
#define EXT_IMU_CTRL2_G          0x11
#define EXT_IMU_OUTX_L_G         0x22
#define EXT_IMU_OUTX_L_A         0x28

typedef struct {
    float x;
    float y;
    float z;
} EXT_IMU_Data;

uint8_t EXT_IMU_Init(void);
void EXT_IMU_Config(void);
void EXT_IMU_ReadAccelerometerData(EXT_IMU_Data *acc_data, uint8_t *raw_data);
void EXT_IMU_ReadGyroscopeData(EXT_IMU_Data *gyro_data, uint8_t *raw_data);

#endif
```

magn_driver.h

```
#ifndef __MAGN_DRIVER_H__
#define __MAGN_DRIVER_H__

#include <stdint.h>
#include "main.h"
#define MAGN_I2C_ADDR          (0x1E << 1)
#define MAGN_WHO_AM_I_REG      0x4F
#define MAGN_WHO_AM_I_VAL      0x40
#define MAGN_CFG_REG_A         0x60
#define MAGN_CFG_REG_B         0x61
```

```
#define MAGN_CFG_REG_C         0x62
#define MAGN_STATUS_REG        0x67
#define MAGN_OUTX_L_REG        0x68

typedef struct {
    float x;
    float y;
    float z;
} MAGN_Data;

uint8_t MAGN_Init(void);
void MAGN_Config(void);
void MAGN_ReadMagnetometerData(MAGN_Data *magn_data, uint8_t *raw_data);

#endif
```

In the `main()` function, dedicated BLE commands and memory buffers are defined to support sensor acquisition and communication.

```
#define BLE_CMD_START_STREAMING   'S'
#define BLE_CMD_STOP_STREAMING    'P'
.
.
.
// ================================================================================
// EXTERNAL IMU
// ================================================================================
// Main sensor used by the current head-controller application.
static EXT_IMU_Data ext_accelerometer_data;
static EXT_IMU_Data ext_gyroscope_data;
uint8_t raw_ext_accelerometer[6] = {0};
uint8_t raw_ext_gyroscope[6] = {0};
// ================================================================================
// MAGNETOMETER
// ================================================================================
// Kept active but transmitted at reduced rate to reduce BLE traffic and power.
static MAGN_Data magnetometer_data;
uint8_t raw_magnetometer[6] = {0};
uint8_t magn_divider = 0;
```

After peripheral initialization, the firmware verifies the presence of the IMU and magnetometer through their identification registers. If detected, the sensors are configured; otherwise, an error is signaled through the red LED.

```
if (EXT_IMU_Init() == 1){
    EXT_IMU_Config();
} else {
    LED_On(LED_RED);
}
if (MAGN_Init() == 1){
    MAGN_Config();
} else {
      LED_On(LED_RED);
}
LED_Off(LED_RED);
```

The firmware includes a command-based finite-state machine with waiting and streaming states. This structure was originally introduced to start and stop transmission from the smartphone and to enable low-power waiting through __WFI(). However, for the final application, continuous streaming was adopted because it proved simpler and more reliable. The command-based structure was retained in the codebase as a possible future enhancement for power-aware operation.

```
while (1){
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
      switch (current_state) {
          case STATE_WAIT_BLE_COMMAND: {
              // Waiting for the smartphone app to
                 send the command 'S'.
              // Timer is stopped and no BLE data
                  packets
                 are transmitted.
              LED_Off(LED_GREEN);
              if (streaming_enabled){
                  Start_Streaming();
              }
              break;
          } case STATE_STREAMING: {
              if (!streaming_enabled) {
                  Stop_Streaming();
                  break;
              } if (sample_ready) {
                  sample_ready = 0;
                  Process_Sensor_Sample();
              }
              break;
          }
      }
      // Low-power wait-for-interrupt.
      // The MCU sleeps between timer/UART/GPIO
         intrerrupts.
      __WFI();

    /* USER CODE END 3 */
  }
```

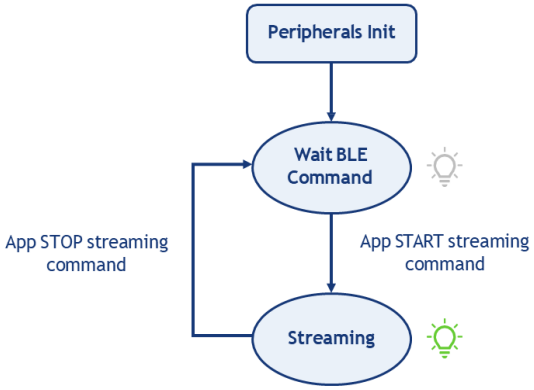


Figure 12: Firmware finite state machine

When a start command is received, the firmware enters the streaming state, enables the status LED, and starts periodic acquisition after a short stabilization delay.

```
static void Start_Streaming(void) {
    sample_ready = 0;
    magn_divider = 0;

    streaming_enabled = 1;
    current_state = STATE_STREAMING;

    LED_On(LED_GREEN);

    // Small stabilization delay after BLE command 'S'
    HAL_Delay(300);

    HAL_TIM_Base_Start_IT(&htim2);
}
```

The acquisition process is driven by TIM2, configured to generate periodic interrupts every 10 ms. Each timer event notifies the main application that a new sensor acquisition cycle must be executed.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim == &htim2) {
        sample_ready = 1;
    }
}
```

At each acquisition cycle, accelerometer and gyroscope measurements are acquired, formatted, and prepared for wireless transmission.

```
static void Process_Sensor_Sample(void) {
    // Read only the sensors needed by the current head-controller app.
```

```
    // Magnetometer is temporarily disabled to reduce BLE load and avoid
    // connection instability during streaming startup.

    EXT_IMU_ReadAccelerometerData(&ext_accelerometer_data, raw_ext_accelerometer);

    EXT_IMU_ReadGyroscopeData(&ext_gyroscope_data, raw_ext_gyroscope);

    BLE_SendPacket(DATA_TYPE_EXT_IMU_ACCELERATION, raw_ext_accelerometer);

    BLE_SendPacket(DATA_TYPE_EXT_IMU_GYROSCOPE, raw_ext_gyroscope);
}
```

Raw IMU data are read through I²C, converted into physical quantities using the sensor sensitivity factors, and prepared for transmission.

```
void EXT_IMU_ReadAccelerometerData(EXT_IMU_Data *acc_data, uint8_t *raw_data) {
    int16_t raw_x, raw_y, raw_z;

    HAL_I2C_Mem_Read(&hi2c3, EXT_IMU_I2C_ADDR, EXT_IMU_OUTX_L_A, I2C_MEMADD_SIZE_8BIT, raw_data, 6,
HAL_MAX_DELAY);

    raw_x = (int16_t)(raw_data[1] << 8 | raw_data[0]);
    raw_y = (int16_t)(raw_data[3] << 8 | raw_data[2]);
    raw_z = (int16_t)(raw_data[5] << 8 | raw_data[4]);

    acc_data->x = raw_x * acc_sensitivity;
    acc_data->y = raw_y * acc_sensitivity;
    acc_data->z = raw_z * acc_sensitivity;
}

void EXT_IMU_ReadGyroscopeData(EXT_IMU_Data *gyro_data, uint8_t *raw_data){
    int16_t raw_x, raw_y, raw_z;

    HAL_I2C_Mem_Read(&hi2c3, EXT_IMU_I2C_ADDR, EXT_IMU_OUTX_L_G, I2C_MEMADD_SIZE_8BIT, raw_data, 6,
HAL_MAX_DELAY);

    raw_x = (int16_t)(raw_data[1] << 8 | raw_data[0]);
    raw_y = (int16_t)(raw_data[3] << 8 | raw_data[2]);
    raw_z = (int16_t)(raw_data[5] << 8 | raw_data[4]);

    gyro_data->x = raw_x * gyro_sensitivity;
    gyro_data->y = raw_y * gyro_sensitivity;
    gyro_data->z = raw_z * gyro_sensitivity;
}
```

Sensor measurements are encapsulated into custom BLE packets and transmitted through the RN4871 module.

```
void BLE_SendPacket(BLE_DataType ble_data_type, uint8_t* data_buffer) {
    uint8_t ble_packet[PACKET_LENGTH];

    // Initialize the packet buffer
    ble_packet[0] = '{';
    ble_packet[PACKET_LENGTH - 1] = '}';
    for (uint8_t i = 1; i < PACKET_LENGTH - 1; i++) {
        ble_packet[i] = 0;
    }

    // Byte 1: Data type identifier
    switch (ble_data_type) {
    case DATA_TYPE_IMU_ACCELERATION:
        ble_packet[1] = 'A';
        break;

    case DATA_TYPE_IMU_GYROSCOPE:
        ble_packet[1] = 'G';
        break;

    case DATA_TYPE_EXT_IMU_ACCELERATION:
        ble_packet[1] = 'a';
        break;

    case DATA_TYPE_EXT_IMU_GYROSCOPE:
```

```
        ble_packet[1] = 'g';
        break;

    case DATA_TYPE_MAGNETOMETER:
        ble_packet[1] = 'M';
        break;

    default:
        ble_packet[1] = 'U';
        break;
}

    // Bytes 2-4: The 32-bit value, packed in big-endian format
    ble_packet[2] = data_buffer[0]; // X Axis LSB
    ble_packet[3] = data_buffer[1]; // X Axis MSB
    ble_packet[4] = data_buffer[2]; // Y Axis LSB
    ble_packet[5] = data_buffer[3]; // Y Axis MSB
    ble_packet[6] = data_buffer[4]; // Z Axis LSB
    ble_packet[7] = data_buffer[5]; // Z Axis MSB

    // Send the complete packet over UART
    BLE_SendData(ble_packet, sizeof(ble_packet));
}
```

A stop command terminates the streaming session. The acquisition timer is disabled, data transmission stops, and the system returns to the waiting state.

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart == &huart3) {
        if (ble_rx_byte == BLE_CMD_START_STREAMING) {
            streaming_enabled = 1;
        } else if (ble_rx_byte == BLE_CMD_STOP_STREAMING) {
            streaming_enabled = 0;
        }

        HAL_UART_Receive_IT(&huart3, &ble_rx_byte, 1);
    }
}

static void Stop_Streaming(void) {
    streaming_enabled = 0;
    sample_ready = 0;

    HAL_TIM_Base_Stop_IT(&htim2);

    current_state = STATE_WAIT_BLE_COMMAND;

    LED_Off(LED_GREEN);
}
```

The firmware architecture was designed with future extensibility in mind. Support for magnetometer acquisition has been implemented and retained within the codebase for future developments, although magnetometer data are not currently transmitted to the smartphone application. The current head-control system relies exclusively on accelerometer and gyroscope measurements. Future developments may focus on integrating magnetometer data and advanced sensor-fusion techniques to improve orientation estimation. At present, BLE packets transmit raw accelerometer and gyroscope measurements, while the conversion performed on the MCU is mainly used for debugging purposes. Future versions could move more processing onboard and transmit higher-level information instead of raw sensor data.

5) **Description of any optional improvements,** e.g., design of a smart eyewear frame, development of an improved smartphone app (maximum 250 words + figures).

Pasquale Ludovico Ignarra (20%), Sofia Capozzi (5%), Fabio Cambedda (5%), Andrea Toselli (30%), Alessandro Gabriela Lima (90%)

Head cursor app

A custom smartphone application was developed in Flutter to provide real-time sensor monitoring and head-based interaction.

The app manages BLE communication through a dedicated connection layer, which scans nearby devices, filters the wearable device, connects to it, and subscribes to the RX characteristic.
Received data are reconstructed as fixed-length 20-byte packets, validated through start and end delimiters, and identified by a data-type byte corresponding to accelerometer, gyroscope, or magnetometer measurements.

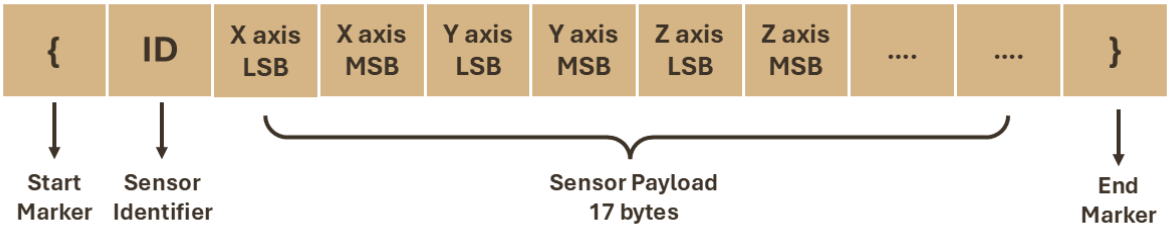


Figure 13: BLE packet structure

First, a scan page with available devices for connection is displayed.

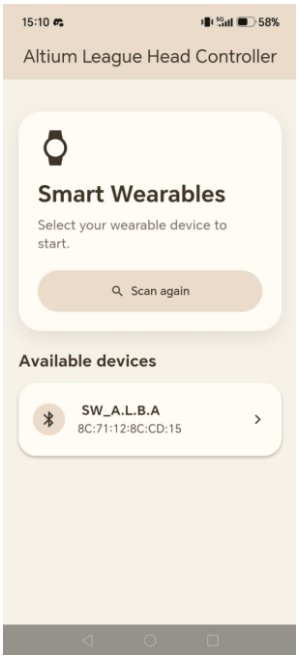


Figure 14: Connection page

After connection, the application provides three main sections: **cursor control**, **sensor graphs**, and **target test**.

The **cursor control** page displays the IMU-based head-controlled cursor and implements dwell-based selection, allowing touchless interaction.

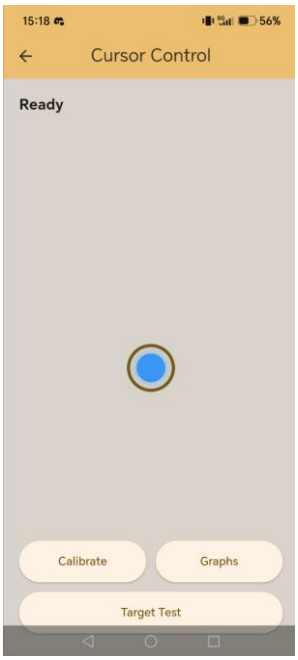


Figure 15: Cursor control page

The **sensor graphs** page decodes the received packets, converts the raw values using the proper sensitivities, applies a low-pass filter, and displays live x, y, and z charts for each sensor. As specified in the code in the previous section, in the actual app implementation only the IMU data are processed and sent to the app, thus only the accelerometer and gyroscope data are plotted.

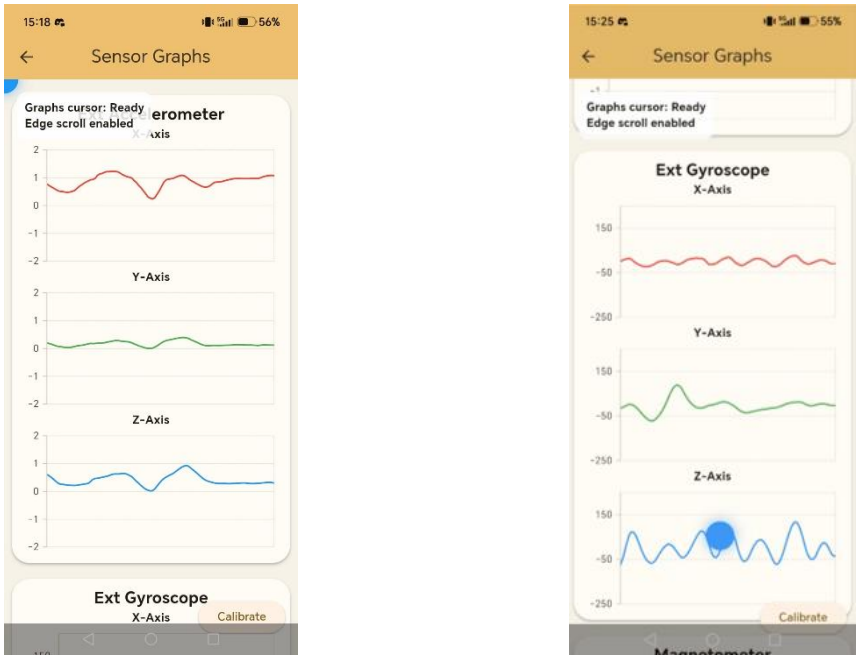


Figure 16: Sensor graphs page

The **target test page** evaluates the cursor performance by recording hits, attempts, accuracy, and average hit time.

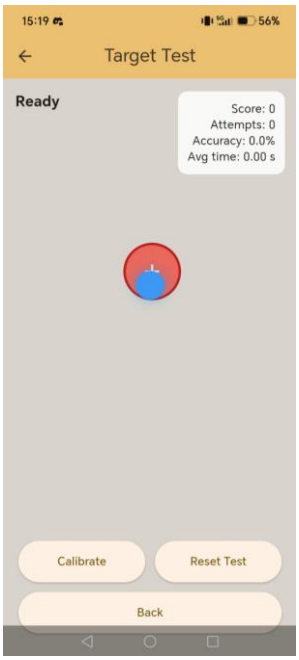


Figure 17: Target test page

Furthermore, gesture-based page navigation was implemented by processing accelerometer and gyroscope measurements. Cooldown intervals and time-window constraints were introduced to improve robustness and minimize false detections. Upward and downward head tilts are mapped to page scrolling actions, whereas left and right head rotations are used to navigate between pages.
The functionality is demonstrated in the video provided at the attached link: video_app.mp4

6) **Final results and discussion** (maximum 250 words + figures and graphs).

Pasquale Ludovico Ignarra (60%), Sofia Capozzi (10%), Fabio Cambedda (10%), Andrea Toselli (5%), Alessandro Gabriela Lima (10%)

Conclusions and next steps

The developed system successfully demonstrates the feasibility of using a wearable inertial probe integrated into smart glasses as a motion-based human–machine interface. The combination of the custom PCB, embedded firmware, BLE communication, and smartphone application resulted in a fully functional proof of concept capable of acquiring head-motion data in real time and translating them into cursor movements and navigation commands. Experimental testing showed that the application provides responsive cursor control and reliable gesture recognition, as shown in the attached video. The implemented filtering techniques, together with cooldown intervals and time-window constraints, significantly reduce false detections and cursor oscillations, resulting in a stable and intuitive user experience. Furthermore, the target-test functionality confirmed the capability of the system to perform accurate pointer positioning and basic interaction tasks without requiring touch input.

Despite these encouraging results, several opportunities for future improvements remain. One important aspect concerns power-consumption optimization. Future firmware versions could exploit the advanced low-power features of the STM32U575, selectively disabling non-essential peripherals and reducing the overall energy consumption of the device.
Another possible development is the execution of part of the data processing directly on the microcontroller. This would reduce BLE traffic, lower radio activity, and extend battery lifetime. Finally, sensor-fusion techniques combining accelerometer, gyroscope, and magnetometer measurements could be integrated to

improve orientation estimation, reduce drift effects, and further enhance the robustness of the motion-based cursor control system.

7) **Attachments**: Probe PCB project, Microcontroller firmware project, any additional designed materials (e.g., 3D design of the eyewear frame, smartphone app…).

The **attachments files** are provided at the following **link**: swdp_project_attachments

8) **Short Video Demo**: maximum 3 minutes video (the link to the video should be provided in the report

The functionality of the app is demonstrated in the **video** provided at the attached link: video_app.mp4

Note: At the beginning of Sections 2–6, list the names of the contributors along with their percentage of contribution (100% indicates that the student participated in all stages of the design and report development).

Example: Maria Bianchi (60%), Mario Rossi (30%), John Smith (100%).